@2025

:: OWASP Top 10 2025: Application Design Flaws

Got it — here's a **short, clean version** you can use for notes/writeup 👇

OWASP Top 10 (2025) – Architecture & Design

Setup

- Start machine in TryHackMe
- Use AttackBox or Kali VPN
- Access via target IP

1 :: AS02: Security Misconfiguration

Meaning: Wrong system settings

Examples:

- Default passwords
- Open ports
- Debug mode ON

Find:

```
nmap -sV <ip>
```

```
gobuster dir -u http://<ip>
```

Fix:

- Disable unused services
- Strong passwords
- Proper config

2 :: AS03: Supply Chain Failures

Meaning: Unsafe third-party libraries

Examples:

- Outdated packages
- Vulnerable dependencies

Find:

```
npm audit
```

```
pip list
```

Fix:

- Update packages
- Use trusted sources

3 :: AS04: Cryptographic Failures

Meaning: Weak encryption

Examples:

- MD5, SHA1
- Plaintext passwords
- No HTTPS

Find:

- Check hashes
- Look for HTTP

Fix:

- Use bcrypt/Argon2
- Enable HTTPS

4 :: AS06: Insecure Design

Meaning: Bad application logic

Examples:

- No rate limit
- IDOR (`/user/123 → /user/124`)

Find:

- Change IDs
- Try brute force

Fix:

- Proper auth & access control
- Rate limiting

Quick Summary

- AS02 → Misconfig
- AS03 → Dependencies
- AS04 → Crypto issues
- AS06 → Design flaws

If you want, I can make it even **1-page exam cheat sheet** 👍

Here's a **simple + clear explanation** of everything you posted 👇

AS02 :: Security Misconfigurations (Easy Explanation)

What It Is

This is when a system is **set up wrongly**, not coded wrongly.

👉 Think like this:

- The app is *built correctly*
- But the **settings are unsafe**

So attackers don't "hack the code" —
they just **take advantage of bad setup**

⚠ Why It Matters

Even a small mistake can:

- Leak sensitive data
- Give admin access
- Let attackers enter the system easily

☞ Modern apps are complex (cloud, APIs, servers)

So **one mistake = full system compromise**

Real Example (Uber 2017)

- A storage bucket was **publicly accessible**
- No login required ×
- Attackers downloaded user data directly

☞ Lesson:

No hacking needed — just access

Common Mistakes

1. Default Credentials

- admin/admin
- root/password

☞ Easy login for attackers

2. Unnecessary Open Services

- Ports/services running but not needed

☞ Attackers scan and exploit them

3. Cloud Misconfiguration

- Public storage (AWS S3, Azure, GCP)
- Wrong permissions

☞ Anyone can access files

4. Missing Authentication

- APIs without login

- No authorization checks

👉 Anyone can access data

5. Verbose Errors

- Shows:

  # File paths

  # Database queries

  # Stack traces

👉 Gives attackers internal info

6. Outdated Software

- Old frameworks with known bugs

👉 Easy to exploit

7. Exposed Endpoints

- `/admin`
- `/debug`
- `/api/users`

👉 No protection → direct access

🛡 How To Prevent

- Change default passwords
- Disable unused services
- Use strong authentication
- Hide error details
- Update software regularly
- Restrict access (least privilege)
- Check cloud permissions
- Secure APIs properly

👉 Basically:

Lock everything that shouldn't be public

Challenge Explanation (IMPORTANT)

🔍 Task

Go to:

```
MACHINE_IP:5002
```

👉 There is a **User Management API**

Hint Meaning

"developers left too many traces"

👉 This usually means:

- Debug info is visible
- API endpoints are exposed
- No proper authentication

What You Should Do

1. Check API Endpoints

Try:

```
/api/user/1
```

```
/api/user/2
```

```
/api/users
```

👉 Look for:

- User data
- IDs
- Emails

2. Test Without Login

- Can you access data directly?
  ☞ If yes → misconfiguration

3. Change IDs (IDOR style)

`/api/user/123 → /api/user/124`

☞ If it shows other users → vulnerability

4. Look for Debug Info

- Error messages
- Hidden routes
- Developer comments

5. Use Tools (Optional)

```
curl http://<ip>:5002/api/user/1
```

Goal

Find:

- Exposed data
- Weak API protection
- Hidden endpoints

☞ Basically:

“What did the developer forget to secure?”

Final Understanding

☞ Security Misconfiguration =

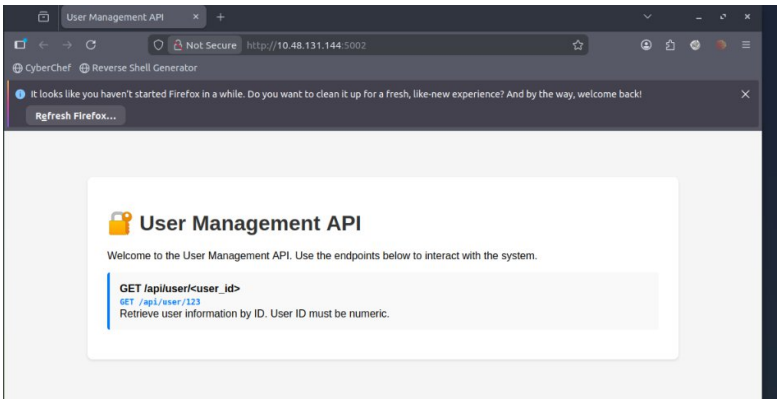
System is open because of bad setup

Not hacking… just:

- Checking

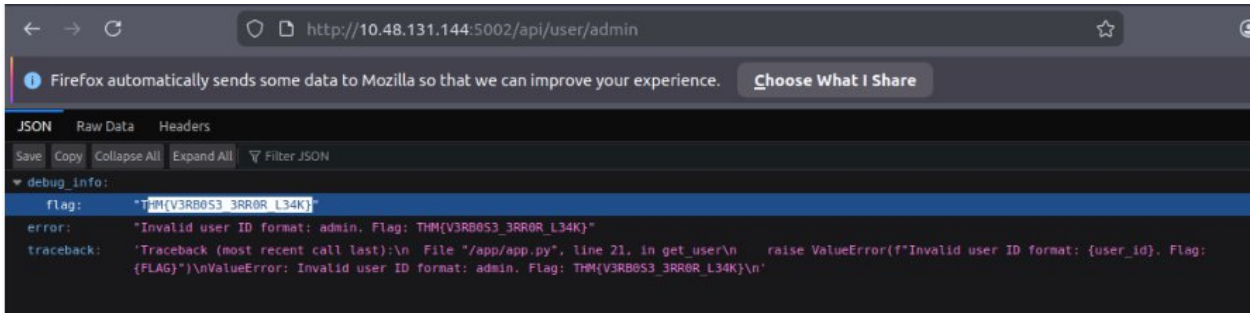
- Accessing
- Observing

Snap:

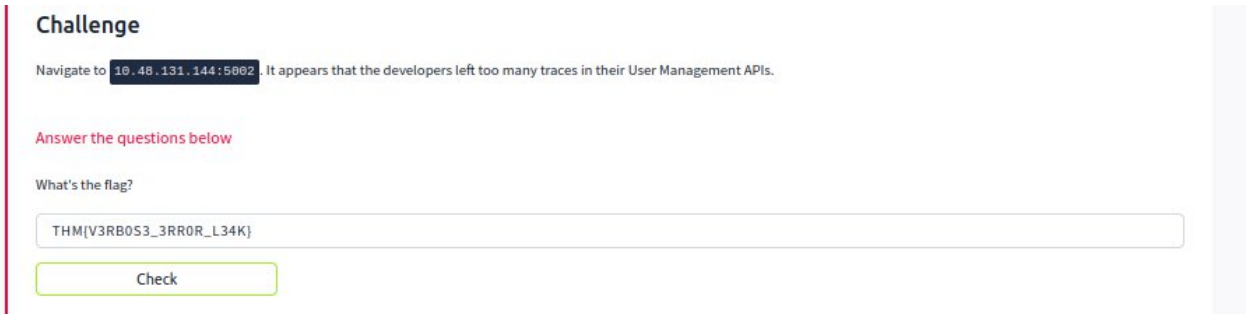


Find the endpoint and change the reference

From ../user/ to ../user/admin



Final flag:



AS03 :: Software Supply Chain Failures

What It Is

Software supply chain failures occur when an application uses **third-party components** (libraries, APIs, tools, or models) that are **compromised, outdated, or not properly verified**.

☞ The vulnerability is **not in your code**, but in the **dependencies you rely on**.

Why It Matters

Modern applications depend heavily on external software.
If even **one dependency is compromised**, attackers can:

- inject malicious code
- steal sensitive data
- gain unauthorized access

☞ Attacks target the **supply chain**, not the application directly.

⚠ Example

The SolarWinds Orion supply chain attack showed how attackers inserted malicious code into a trusted software update, affecting thousands of organizations.

☞ The attack spread through **trusted updates**, not direct hacking.

Common Patterns

- Using unverified or outdated libraries
- Installing updates without validation
- Insecure CI/CD pipelines (build/deploy process)
- No tracking of dependency source (provenance)
- Lack of monitoring after deployment

🛡 How to Prevent

- Verify and use trusted dependencies
- Regularly update and patch components
- Use signed and verified packages
- Secure CI/CD pipelines from tampering
- Track origin and licensing of dependencies
- Monitor applications for unusual behavior

Challenge Insight

If a project uses something like:

`lib/vulnerable_utils.py`

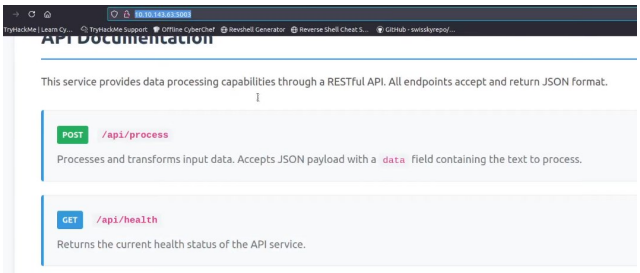
👉 Check for:

- unsafe functions (e.g., `eval`, `exec`)
- outdated or insecure logic
- hidden malicious code

👉 Fix by updating, replacing, or securing the component.

If you want, I can convert this into a **perfect exam answer or README.md format**.

Snap:



Challenge

Navigate to `MACHINE_IP:5003`. The code is outdated and imports an old `lib/vulnerable_utils.py` component. Can you ***debug*** it?

Here is your content formatted cleanly for documentation (**without changing any line**, only structured neatly in Markdown):

Lab: Debugging Vulnerable API

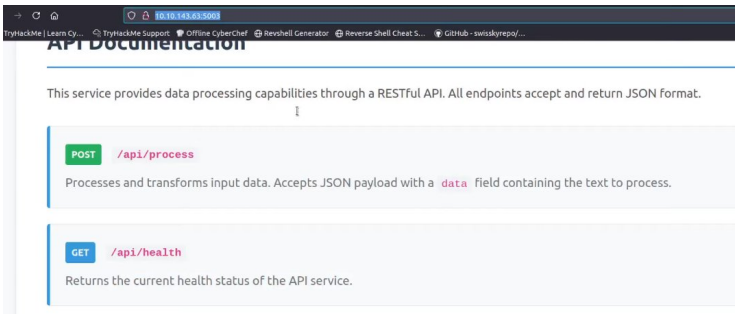
Objective

Identify and exploit a vulnerability in the API running on MACHINE_IP:5003 to retrieve sensitive debug information and the flag.

🔍 Step 1: Initial Enumeration

Open the target:

`http://MACHINE_IP:5003`



You find an API documentation page showing:

`POST /api/process`

`GET /api/health`

👉 Key observation:

- API accepts JSON input
- Main functionality is in `/api/process`

Step 2: Directory Enumeration

Run gobuster:

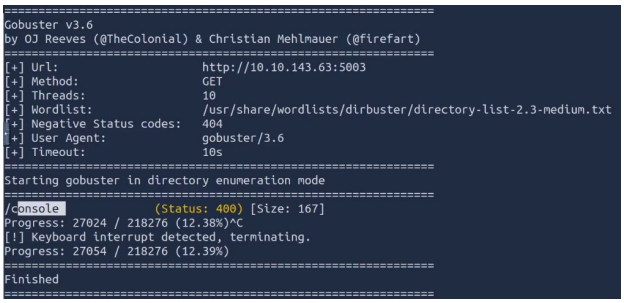
```
gobuster dir -u http://MACHINE_IP:5003 -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

Result:

```
/console (Status: 400)
```

👉 Indicates hidden/debug-related endpoint exists

Snap:



🔍 Step 3: Analyze JavaScript

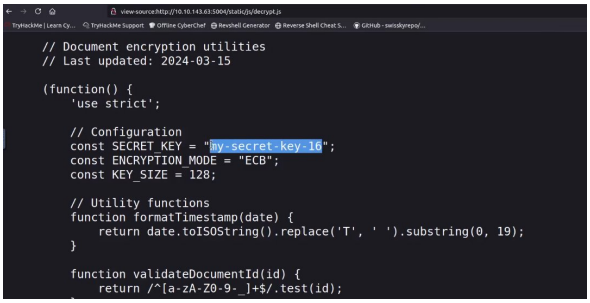
From browser:

`/static/js/decrypt.js`

Important finding:

```
const SECRET_KEY = "my-secret-key-16";

const ENCRYPTION_MODE = "ECB";
```



👉 Weak crypto (ECB) + exposed key → bad practice

👉 Confirms debug/dev code left in production

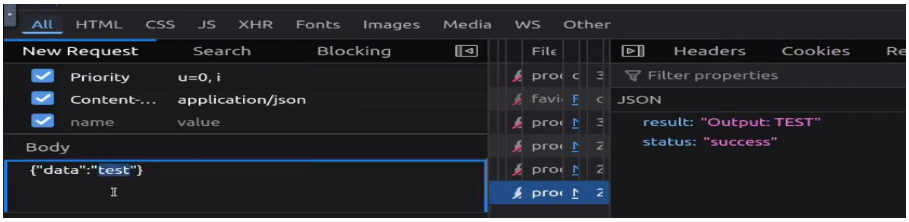
🔍 Step 4: Test API Endpoint

Intercept request using Burp Suite

Request:

`POST /api/process HTTP/1.1`

`Content-Type: application/json`

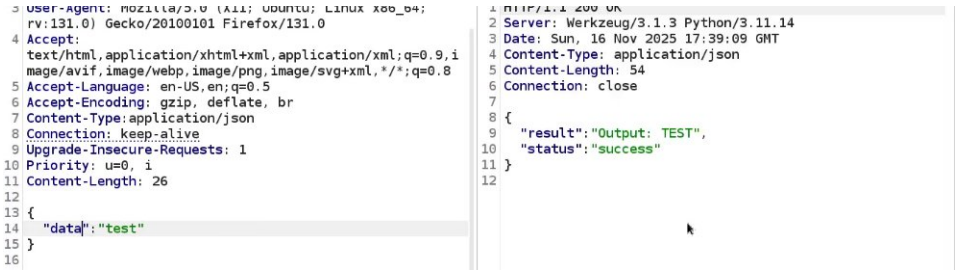



```
{
  "data": "test"
}
```

Response:

```
{
  "result": "Output: TEST",
  "status": "success"
}
```

In burp,



Endpoint works normally

⚠ Step 5: Identify Vulnerability (Code Analysis)

From given code:

```
if data == 'debug':
    return jsonify(debug_info())
```

Vulnerability:

- No authentication
- Direct access to `debug_info()`
- Sensitive info exposure

☞ This is Improper Debug Mode Exposure

Snap:

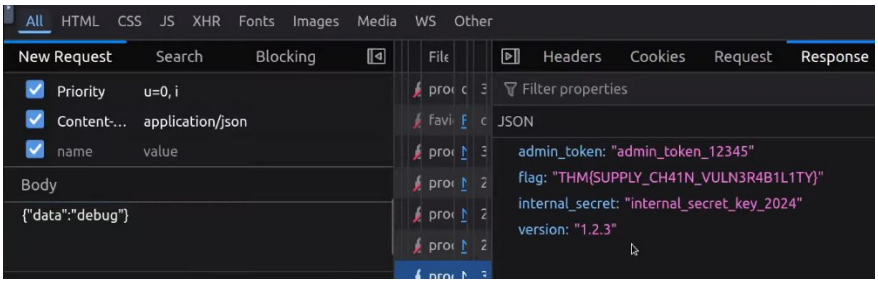
```
# Check for debug mode
if data == 'debug':
    return jsonify(debug_info())
```

Step 6: Exploitation

Modify request in Burp Repeater:

```
POST /api/process HTTP/1.1
Content-Type: application/json
{
  "data": "debug"
}
```

Snap:

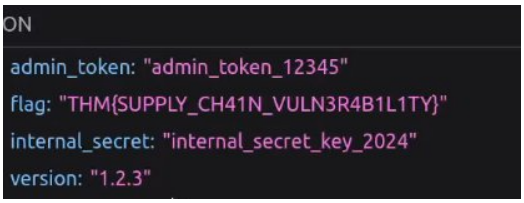


Step 7: Capture Sensitive Data

Response:

```
{
  "admin_token": "admin_token_12345",
  "flag": "THM{SUPPLY_CH41N_VULN3R4B1L1TY}",
  "internal_secret": "internal_secret_key_2024",
  "version": "1.2.3"
}
```

Snap:



Final Flag

`THM{SUPPLY_CH41N_VULN3R4B1L1TY}`

Root Cause Analysis

Issues Identified:

1. Debug functionality exposed
   - Triggered via user input (`data == 'debug'`)
2. Untrusted local library

   * `from vulnerable_utils import debug_inf`

   * Possible supply chain vulnerability

3. No authentication/authorization
   - Anyone can access debug data

🛡 Mitigation

- Remove debug functionality from production
- Protect debug endpoints with authentication
- Avoid importing unverified local libraries
- Never expose secrets in API responses
- Use environment variables for sensitive data

Conclusion

The application is vulnerable due to:

- Improper debug handling
- Unsafe third-party/local library usage

By sending:

```
{"data":"debug"}
```

we successfully triggered hidden debug functionality and retrieved the flag.

AS04 :: Cryptographic Failures

What It Is

- Encryption is **missing, weak, or misused**
- Sensitive data becomes **accessible to attackers**

⚠ Why It Matters

- Protects passwords, tokens, and personal data
- Failure leads to:

  * Data leaks

  * Account takeover

Common Issues

- Weak algorithms (MD5, SHA-1, ECB)
- Hardcoded keys in code
- No encryption (HTTP)
- Poor key management (no rotation)
- Invalid/self-signed certificates

Attack Methods

- MITM (intercept communication)
- Brute-force weak keys
- Extracting exposed secrets

Prevention

- Use strong crypto (AES-GCM, TLS 1.3)
- Store keys in secure vaults
- Rotate keys regularly
- Avoid hardcoding secrets

Challenge

Steps

- Open: `http://MACHINE_IP:5004`

- Inspect:
    - JS files , Source code
    - Network tab

- Find:
    - Encrypted data, Hardcoded key
- Decrypt using CyberChef/OpenSSL

snap:

Flag:

THM{CRYPTO_FAILURE_H4RDC0D3D_K3Y}

⚠ Vulnerability

- Hardcoded key exposed

Fix

- Use secure key storage
- Rotate keys
- Do not expose secrets

Here's a **clean, point-wise + slightly detailed (same style as before)** 👇

🏗 Insecure Design

What It Is

- Security flaws built into the **system design or logic**
- Caused by:
  - No threat modelling
  - Wrong assumptions
  - Poor architecture decisions
- **!** Cannot be fixed easily (needs redesign)

⚠ Why It Matters

- Vulnerability exists in **core logic**, not just code
- Leads to:
  - Unauthorized access
  - Data leaks
  - System abuse
- Fixing requires **changing the whole design**

Example

- Clubhouse app:
  - Assumed only mobile app users would access backend
  - No proper authentication
- Result:
  - Anyone could access user data & private conversations

Common Issues (2025)

- Weak business logic (password reset, approvals)
- Wrong assumptions about users or systems
- Systems/components with too much access
- Missing controls for automation/LLMs
- Debug/test features left in production
- No proper threat modelling

Modern Risks (LLM/AI Systems)

- Prompt injection (user input manipulates system)
- Blind trust in model outputs
- Unverified/poisoned models

- No validation of AI decisions

Attack Methods

- Bypassing business logic
- Direct API access (skipping frontend)
- Exploiting missing authentication
- Manipulating inputs/prompts

Secure Design Practices

- Perform threat modelling at every stage
- Validate all inputs & outputs
- Separate user input from system logic
- Apply least privilege (minimal access)
- Enforce authentication & authorization
- Keep sensitive data protected
- Monitor and test for abuse cases
- Require human review for critical actions

Challenge

Objective

- Check if system wrongly assumes **only mobile access**
- Bypass and access it from browser

Steps

🔍 1. Open Target

```
http://MACHINE_IP:5005
```

2. Analyze Behaviour

- Does it:
    - Block browser access?
    - Show mobile-only message?

👉 Indicates **bad design assumption**

3. Bypass Restriction

- Try:
  - Changing **User-Agent** (browser → mobile)
  - Using tools like Burp Suite / DevTools

Example:

```
User-Agent: Android / iPhone
```

🔗 4. Access Backend

- Directly access:
  - API endpoints
  - Hidden routes

👉 Backend may not enforce authentication

Result

Answer the questions below

What's the flag?

THM{1NS3CUR3_D35IGN_4SSUMPT10N}

⚠ Vulnerability

- Assumed only mobile users will access system
- No proper backend security

Fix

- Do not trust frontend restrictions
- Enforce security on backend
- Implement proper authentication
- Validate all client requests

If you want, I can also show **exact Burp Suite steps to bypass User-Agent like in real CTF** 👍